

Geovisualization of OpenStreetMap Data in Berlin

Where are the restaurants, cafés, pharmacies ... and where are the gaps?

Tools: Python · pandas · GeoPandas · OSMnx / Overpass API · Folium · Streamlit · LLM chatbot (API) · GitHub

1. Background

OpenStreetMap (OSM) is the free world map created by volunteers. It contains millions of points of interest (POIs) such as restaurants, cafés, pharmacies, supermarkets or playgrounds. Every object carries key=value tags, for example amenity=restaurant or cuisine=italian.

You receive a starter app (Streamlit) that shows OSM data of Berlin on a map. The user selects in pull-down menus which data source is shown and how it is aggregated and displayed. In addition, the app contains a chatbot that lets the user ask questions about the data in natural language.

Chatbot: ask the data

The chatbot is powered by a large language model (LLM) that is called via an API, for example OpenAI or OpenRouter (also DeepSeek, Mistral, Anthropic or a local Ollama model). You use your own API key.

Example questions: “Which three districts have the most pharmacies per 10,000 residents?” – “Where are gaps?” – “What do the data quality results mean for this conclusion?” The bot answers about the current view; it receives your question and the aggregated numbers and quality report, not the raw OSM points.

The starter app is a starting point, not the goal

The starter app is deliberately generic: every data source, area unit and map type for everybody, technical labels, no meaningful defaults, no focus. Its user experience (UX) is not optimal.

In Part B you are free to choose your own use case (a user and a question) and to optimize the app for it.

Important: OSM is crowd-sourced and its completeness varies. An area with few points may simply be less well mapped. LLM answers can sound convincing and still be wrong. Verify important numbers.

2. Data

Dataset	Content	Source
OSM points of interest	One csv per data source, e.g. all amenity=restaurant in Berlin	OpenStreetMap via Overpass API (OSMnx)
geodata_berlin_plz.csv / _dis.csv	Polygons of postal codes (PLZ) and districts (Bezirk), WKT	Starter package
plz_einwohner.csv	Residents and area per PLZ	suche-postleitzahl.org

config.py defines seven OSM data sources: restaurants, cafés, fast food (amenity=restaurant / cafe / fast_food), pharmacies (amenity=pharmacy), supermarkets (shop=supermarket), playgrounds (leisure=playground) and charging stations (amenity=charging_station). For your own use case you can

add any other tag (wiki.openstreetmap.org, “Map features”). OSM data is under the ODbL; the app shows “© OpenStreetMap contributors”.

3. The starter application

#	Element	Options	Behaviour
1	Data source	All labels from config.py	Decides which file is loaded.
2	Category filter	“All” plus the ten most frequent values of an attribute	Depends on menu 1.
3	Area unit	Postal code (PLZ) / District (Bezirk)	Polygons used for aggregation.
4	Indicator	Number / per km ² / per 10,000 residents	Residents only available per PLZ.
5	Map type	Choropleth / Heatmap / Point map with clusters	Choropleth uses menus 3 and 4.
6	Chatbot	Free text; provider, model and API key in the sidebar	Answers about the current view (core/chat.py, provided).

Below the map: key figures, a data table with CSV download, a data quality report and the attribution.

4. Tasks

Part A – Make the starter app work

The starter package contains the structure and many ready parts. The places you complete are marked with TODO.

- **Task 1 – Download the OSM data.** Complete `download_osm.py`: one file per data source in `datasets/` (OSMnx, `ox.features_from_place`). Convert every object (point or polygon) to one point: centroid in EPSG:25833, back to EPSG:4326. Download once; the app never calls the Overpass API.
- **Task 2 – Spatial join and indicators.** Assign each point to a PLZ and a Bezirk with `geopandas.sjoin` (not via `addr:postcode`, which is often missing). Aggregate per area starting from the polygons, so empty areas get 0. Compute number, number per km² (area in EPSG:25833, never degrees) and number per 10,000 residents; decide how to handle areas with very few residents.
- **Task 3 – Data quality check.** Complete `quality_report()`: number of objects, share without name, near-duplicates, points outside Berlin, plausible coordinates, consistency of `addr:postcode` with your join.
- **Task 4 – Complete and run the app, including the chatbot.** Complete `make_streamlit_app` and `make_map` (three map types, Folium) and run it with the command below. Create an API key at an LLM provider (e.g. OpenAI or OpenRouter), store it in `.streamlit/secrets.toml` (template provided) or enter it in the sidebar, and test the chatbot with three questions. Never write a key into code or push it to GitHub. Data sources and LLM providers are defined in `config.py`.

```
streamlit run main.py
```

Part B – Your own use case

- **Task 5 – Define your use case.** Half a page: who uses the app, which question does this person want to answer, which decision does it support? Examples: a family looking for a district with playgrounds and pharmacies nearby; someone opening a café who looks for a district with few cafés but many residents; a planner checking where charging stations fall behind; a visitor looking for vegetarian or wheelchair-accessible places. Then analyse what in the starter app is confusing, missing or unnecessary for this use.

- **Task 6 - Optimize the app for it.** Make at least three concrete improvements and justify each in one sentence. You are free in how. Ideas: hide menus you do not need, set meaningful defaults and plain-language labels; add other OSM sources or combine several; create an indicator, colour scale or legend that fits your question; change the layout; give the chatbot a tailored role, example questions or more context.
- **Task 7 - Publish and document.** Push code and data to GitHub (short README) and deploy on Streamlit Community Cloud (API key under Settings → Secrets). Write a documentation of about 2 pages (PDF): use case, what you changed and why, the answer to your question with screenshots, limits of the OSM data, and a check of two chatbot answers against your data table.

5. What to hand in

Link to the GitHub repository, link to the deployed app, and the documentation as PDF.

Pitfalls: GeoPandas uses (lon, lat), Folium expects [lat, lon]. Never compute areas or distances in degrees. Central districts are mapped more densely, so a low value is no proof of a lack of supply. Do not send Overpass requests in a loop.